

The κ -Source Framework

Theoretical Formulation, Mock Image Generation, and Hierarchical Extraction

Standalone High-Performance Rust Implementation | Software Version 0.1.0

Abstract

This document provides the formal theoretical definition and practical operating instructions for the **κ -Source Framework**. Under this formalism, any astronomical image is conceptualized as the union of κ -sources ($\kappa \in \{1, 2, \dots, n\}$). We define a κ -source as a coherent cluster of κ constituent point sources within a spatial radius $R \leq R_{\max}$ whose individual flux is below standard detection thresholds ($F_i < 3 \text{ RMS}$), but whose collective integrated flux satisfies $\sum F_i \geq 3 \text{ RMS}$. We present high-performance Rust tools (`kappa_generate.bin` and `kappa_extract.bin`) to synthesize mock FITS images and perform hierarchical extraction from arbitrary FITS files.

1 Theoretical Concept of κ -Sources

1.1 The Fundamental Image Decomposition Principle

In conventional astronomical source detection (e.g., standard thresholding), images are decomposed solely into isolated point-like or extended components exceeding a local detection threshold (typically 3σ or 5σ). Faint sub-threshold components buried in noise are discarded.

Under the **κ -Source Formalism**, an astronomical image $I(x, y)$ is defined as the union of κ -sources:

$$I(x, y) = \mathcal{N}(0, \sigma_{\text{noise}}^2) + \bigcup_{\kappa=1}^n \bigcup_{j=1}^{N_{\kappa}} S_{\kappa,j}(x, y)$$

where κ designates the **multiplicity** (number of constituent point sources) of the source, and N_{κ} is the number of κ -sources of order κ .

1.2 Mathematical Definition of a κ -Source

A κ -Source $C_{\kappa} = \{s_1, s_2, \dots, s_{\kappa}\}$ of multiplicity κ is formed by κ point sources satisfying two strict criteria:

1. **Spatial Coherence Constraint:** All κ constituent point sources reside within a maximum clustering radius $R_{\max}$ from their flux-weighted centroid:

$$\max_{i \in \{1.. \kappa\}} \| \mathbf{x}_i - \mathbf{x}_{\text{cen}} \| \leq R_{\max}$$

2. **Collective Significance & Sub-threshold Condition:**

- For $\kappa \geq 2$ (Multi-component κ -sources): Each constituent subcomponent i is individually below the point-source detection threshold:

$$\forall i \in \{1.. \kappa\}, \quad F_i < 3.0 \times \sigma_{\text{RMS}}$$

while their collective integrated flux satisfies the detection threshold:

$$F_{\text{total}} = \sum_{i=1}^{\kappa} F_i \geq 3.0 \times \sigma_{\text{RMS}}$$

- For $\kappa = 1$ (1-Sources): The single isolated source meets detection on its own:

$$F_1 \geq 3.0 \times \sigma_{\text{RMS}}$$

Multiplicity κ	Individual Flux F_i	Collective Flux $\sum F_i$	Observational Classification
$\kappa = 1$ (1-source)	$F_1 \geq 3\sigma_{\text{RMS}}$	$F_1 \geq 3\sigma_{\text{RMS}}$	Directly detected point source
$\kappa = 2$ (2-source)	$F_i < 3\sigma_{\text{RMS}}$	$\sum F_i \geq 3\sigma_{\text{RMS}}$	Pair of faint subcomponents
$\kappa = 3$ (3-source)	$F_i < 3\sigma_{\text{RMS}}$	$\sum F_i \geq 3\sigma_{\text{RMS}}$	Triplet of faint subcomponents
$\kappa = n$ (n -source)	$F_i < 3\sigma_{\text{RMS}}$	$\sum F_i \geq 3\sigma_{\text{RMS}}$	Cluster of n faint subcomponents

1.3 Hierarchical Extraction Sequence

Extraction proceeds **hierarchically in order of increasing multiplicity**:

1. First, all **1-sources** ($\kappa = 1$) are identified and cataloged.
2. Next, all **2-sources** ($\kappa = 2$) are identified and cataloged.
3. Next, all **3-sources** ($\kappa = 3$), up to $\kappa = n$.

For every extracted κ -source, the following physical properties are measured:

- **Flux-weighted Centroid**:

$$\mathbf{x}_{\text{cen}} = (\bar{x}, \bar{y}) = \left(\frac{\sum_{i=1}^{\kappa} F_i x_i}{\sum_{i=1}^{\kappa} F_i}, \frac{\sum_{i=1}^{\kappa} F_i y_i}{\sum_{i=1}^{\kappa} F_i} \right)$$

- **Characteristic Spatial Extent (Radius)**:

$$R = \max_{i \in \{1.. \kappa\}} \sqrt{(x_i - \bar{x})^2 + (y_i - \bar{y})^2} + \frac{\text{FWHM}}{2}$$

- **Signal-to-Noise Ratio (SNR)**:

$$\text{SNR} = \frac{F_{\text{total}}}{\sigma_{\text{RMS}}}$$

2 Multi-Extension FITS Architecture

All data generated and extracted are stored in standard multi-extension astronomical FITS files:

HDU	Name	Type	Contents
0	PRIMARY	Image HDU	2D Float array ($M \times M$) with metadata (NKAPPA , KAP_MAX , KAP_RAD , DET_SIG , BG_SIGMA)
1	SOURCES	Binary Table	Point sources catalog (ID , X , Y , FLUX , AMPLITUDE , SIGMA , FWHM , KAPPA_ID , KAPPA)
2	KAPPA_SRCS	Binary Table	Extracted κ -sources catalog (KAPPA_ID , KAPPA , CEN_X , CEN_Y , TOTAL_FLUX , RADIUS , SNR , N_MEMBERS)

3 Practical Guide: Generating Mock FITS Images

The tool `kappa_generate.bin` synthesizes an $M \times M$ mock FITS image containing N κ -sources with multiplicity $\kappa \leq \kappa_{\max}$ and cluster radius $R \leq R_{\max}$ on top of Gaussian background noise.

3.1 Command-Line Parameters

Option	Short	Default	Description
<code>--num-kappa</code>	<code>-N</code>	50	Total number of κ -sources to produce (N)
<code>--size</code>	<code>-M</code>	4096	Image grid dimension M ($M \times M$ pixels)
<code>--max-kappa</code>	<code>-k</code>	5	Maximum multiplicity bound ($1 \leq \kappa \leq \kappa_{\max}$)
<code>--max-radius</code>	<code>-r</code>	25.0	Maximum spatial cluster radius in pixels ($R_{\max}$)
<code>--detection-sigma</code>	<code>-s</code>	3.0	Collective flux threshold ($\sum F_i \geq S \times \text{RMS}$)
<code>--noise-sigma</code>		1.0	Background Gaussian noise standard deviation (RMS)
<code>--fwhm</code>		10.0	Gaussian PSF Full Width at Half Maximum in pixels
<code>--output</code>	<code>-mock_kappa_image.fits</code>		Output FITS file path

3.2 Usage Examples

```
# 1. Generate 50 kappa-sources (kappa <= 5, radius <= 25 px) on a 4096 x 4096 grid:
./kappa_generate.bin -N 50 -M 4096 -k 5 -r 25.0 -s 3.0 --output mock_kappa.fits

# 2. Generate 100 kappa-sources with custom noise RMS = 2.0 and detection = 5xRMS:
./kappa_generate.bin -N 100 -M 4096 -k 6 -r 30.0 --noise-sigma 2.0 -s 5.0 -o sim.fits
```

4 Practical Guide: Extracting κ -Sources from FITS

The tool `kappa_extract.bin` ingests any arbitrary 2D FITS file (real or simulated) and extracts the κ -source hierarchy.

4.1 Extraction Algorithm Pipeline

- Noise Background Ingestion:** Computes image median and Median Absolute Deviation ($\sigma = 1.4826 \times \text{MAD}$).
- Gaussian Matched Filtering:** Convolves the 2D image with the PSF kernel to maximize the SNR of point-source features and eliminate single-pixel noise fluctuations.
- Subcomponent Peak Finding:** Detects local intensity peaks with sub-pixel quadratic centroid refinement and aperture flux integration.
- Hierarchical Graph Clustering:** Groups subcomponents within distance $R_{\max}$, validates $\sum F_i \geq 3 \text{ RMS}$, verifies individual subcomponent limits, and sorts from $\kappa = 1$ upwards.
- Catalog Export:** Produces `<input>.extracted.fits` and `<input>.extracted.reg` (DS9 region overlay).

4.2 Command-Line Parameters

Option	Short	Default	Description
<code><INPUT></code>		(required)	Path to input FITS image (e.g. <code>test.fits</code>)

<code>--max-kappa</code>	<code>-k</code>	0 (all)	Maximum multiplicity upper limit (e.g. <code>-k 3</code> restricts to $\kappa \leq 3$)
<code>--detection-sigma</code>	<code>-s</code>	3.0	Collective flux detection threshold ($\geq S \times \text{RMS}$)
<code>--cluster-radius</code>	<code>-r</code>	25.0	Maximum clustering radius in pixels from centroid
<code>--fwhm</code>		10.0	Estimated PSF FWHM in pixels
<code>--peak-snr</code>		2.5	Matched-filter peak SNR threshold for candidate peaks
<code>--output</code>	<code>-o<in>.extracted.fits</code>		Output FITS catalog path

4.3 Usage Examples

```
# 1. Standard extraction with 3xRMS threshold and 25 px radius:
./kappa_extract.bin test.fits -s 3.0 -r 25.0

# 2. Extract only up to 3-sources (kappa <= 3) with 5xRMS detection threshold:
./kappa_extract.bin test.fits -k 3 -s 5.0 -r 30.0 -o catalog.fits
```

5 Visualization with SAOImage DS9 & Python

5.1 DS9 Color-Coded Overlay

Both `kappa_generate.bin` and `kappa_extract.bin` automatically export companion `.reg` files color-coded by multiplicity hierarchy:

- **Green circles:** 1-Sources ($\kappa = 1$)
- **Yellow circles:** 2-Sources ($\kappa = 2$)
- **Orange circles:** 3-Sources ($\kappa = 3$)
- **Red circles:** $\kappa \geq 4$ -Sources
- **Dashed white circles:** Constituent subcomponent peaks

Open the image and region overlay with one command:

```
ds9 test.fits -regions test.extracted.reg -scale zscale -zoom to fit
```

5.2 Inspecting Catalogs in Python (Astropy)

```
from astropy.io import fits
import numpy as np

with fits.open("test.extracted.fits") as hdul:
    hdul.info()

    # 1. Access extracted kappa-sources catalog
    ktable = hdul["KAPPA_SRCS"].data
    print(f"Total kappa-sources extracted: {len(ktable)}")
    for row in ktable[:5]:
        print(f"ID={row['KAPPA_ID']}: kappa={row['KAPPA']}, Pos=({row['CEN_X']:.1f}, {row['CEN_Y']:.1f}), Flux={row['TOTAL_FLUX']:.3f}, SNR={row['SNR']:.1f}")
```

